

02244 Logic for Security Information Flow Week 10: Myers' Approach

Sebastian Mödersheim

April 20, 2026

Challenge

Let $\gamma = [x \mapsto \text{Low}, y \mapsto \text{High}]$ and the context $\text{ctx} = \text{Low}$

- Is the program $S = (\text{while } y > 0 \text{ do } y := y + 1) ; x := 2$ secure?

Use Volpano's typing rules to determine if S is secure

- Do you agree with the result of the type system?

Declassification

Example: Login-Program

```
pinfo = record [name,password:string{H}]
check_pw(db:array[pinfo]{H},
         name:string{L}, password:string{H})
  returns ret:bool{L}
i: int{L} :=0;
match: bool{???} :=false;
while (i<db.length) do
  if db[i].name=name && db[i].password=password
  then match:=true
  i:=i+1
ret:=match
```

What label should match have?

Declassification

What we **cannot** do with classical information flow: release some information that is depending on something classified.

- Log in: the password data-base is secret, but the information whether you have entered the right password is not.
- Result of an election: the votes are secret/private, but the result is public.
- Medical database: the medical records are secret, but they may be released to a researcher after personal information is removed.
- Electronic Auction: the max bids of customers is secret at first, but then during bidding they are partially revealed.

Declassification

What we **cannot** do with classical information flow: release some information that is depending on something classified.

- Log in: the password data-base is secret, but the information whether you have entered the right password is not.
- Result of an election: the votes are secret/private, but the result is public.
- Medical database: the medical records are secret, but they may be released to a researcher after personal information is removed.
- Electronic Auction: the max bids of customers is secret at first, but then during bidding they are partially revealed.

We thus want a mechanism to explicitly **declassify** information in a **fine-grained** way.

Declassification

Information Flow gives you a strong guarantee:

An intruder who can only observe the low variables, cannot learn anything about the high variables.

We have logically formalized this guarantee in the previous lecture: non-interference.

Declassification means that you will lose this guarantee.

- Log in: an intruder can do a guessing attack
- Result of an election: you learn a bit about the votes
- Medical database: an intruder may be able to reconstruct some information about the patients.
- Electronic Auction: an intruder learns a bit about the bids

Giving up Control?

If you give an intruder (dishonest person) some information, you lose all control over it. But the world is more complicated.

Giving up Control?

If you give an intruder (dishonest person) some information, you lose all control over it. But the world is more complicated.

Consider a large organization like a hospital:

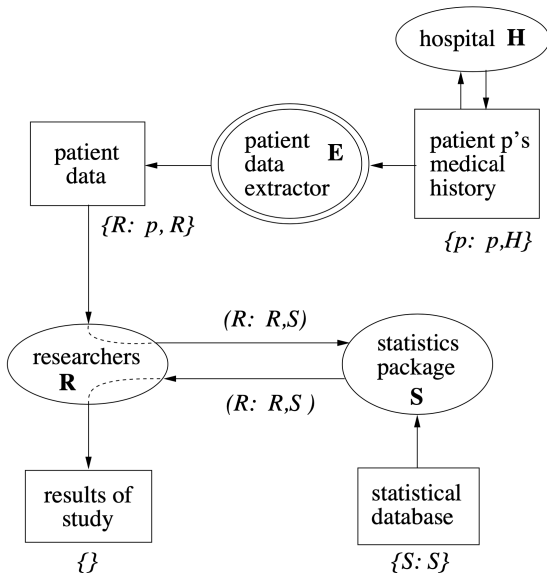
- Even though the hospital itself is honest, it may run some systems that are not secure.
- Systems that are designed by honest people could have bugs.
- When declassifying information, you may not want to give permission to use the data **arbitrarily**.
 - ★ There may be a usage policy about using the declassified data, and compliance may be required by law, e.g. GDPR.
 - ★ Similarly, release of data may be subject to usage policy by a contract, e.g., the researchers must make a contract with the hospital to get access to patient data.
- How to formally specify such policies and automatically prove compliance?

The Decentralized Label Model

Andrew C. Myers and Barbara Liskov: *A Decentralized Model for Information Flow Control*, ACM Symposium on Operating System Principles, 1997 [1].

- ① **Security lattice**: instead of high and low we have more complicated security labels:
 - ★ A set of **owners**: participants or roles who own the respective data
 - ★ An owner can say who can **read** the data.
 - ★ You can only read data if **all** owners have allowed it.
- ② We will see later how to compare security labels.
Except for declassification this is standard information flow à la Denning or Volpano.
- ③ **Declassify** limited: an owner can only relax **their own constraint**.
- ④ Programs can act **on behalf** of an owner and thus declassify, but this forces programmer to make every declassification explicit, so one does not accidentally **forget** about the rights of some owner.

Hospital Example



Overview

① Security lattice:

- ★ A set of **owners**: participants or roles who own the respective data
- ★ An owner can say who can **read** the data.
- ★ You can only read data if **all** owners have allowed it

② Defining $\sqsubseteq$, $\sqcup$, $\sqcap$.

Except for declassification this is standard information flow à la Denning from the last lecture.

③ **Declassify**: an owner can relax their own constraint.

④ Programs can act **on behalf** of an owner and thus declassify.

1. The Security Lattice

We have the security framework $(P \hookrightarrow \text{PowerSet}(P), \sqsubseteq, \sqcup, \sqcap)$ where

- $P \hookrightarrow \text{PowerSet}(P)$ is the set of all **partial mappings** from P to $\text{PowerSet}(P)$.
 - ★ $s_1 = \{A : \{A, B\}\}$
 - ★ $s_2 = \{B : \{A\}\}$
 - ★ $s_3 = \{B : \{A\}, A : \{\}\}$
- For a label s we define $\text{Owners}(s) = \text{Domain}(s)$
 - ★ $\text{Owners}(s_1) = \{A\}$
 - ★ $\text{Owners}(s_2) = \{B\}$
 - ★ $\text{Owners}(s_3) = \{A, B\}$

1. The Security Lattice

We have the security framework $(P \hookrightarrow \text{PowerSet}(P), \sqsubseteq, \sqcup, \sqcap)$ where

- $P \hookrightarrow \text{PowerSet}(P)$ is the set of all **partial mappings** from P to $\text{PowerSet}(P)$.

- ★ $s_1 = \{A : \{A, B\}\}$

- ★ $s_2 = \{B : \{A\}\}$

- ★ $s_3 = \{B : \{A\}, A : \{\}\}$

- For a label s we define $\text{Owners}(s) = \text{Domain}(s)$
- For a security label s and principal p define

$$\text{Readers}(s, p) = \begin{cases} s(p) & \text{if } p \in \text{Owners}(s) \\ P & \text{if } p \notin \text{Owners}(s) \end{cases}$$

- ★ $\text{Readers}(s_1, A) = \{A, B\}$

- ★ $\text{Readers}(s_2, B) = \{A\}$

- ★ $\text{Readers}(s_3, B) = \{A\}$

- ★ $\text{Readers}(s_3, A) = \{\}$

- ★ $\text{Readers}(s_3, C) = P$ (everybody)

1. The Security Lattice

We have the security framework $(P \hookrightarrow \text{PowerSet}(P), \sqsubseteq, \sqcup, \sqcap)$ where

- $P \hookrightarrow \text{PowerSet}(P)$ is the set of all **partial mappings** from P to $\text{PowerSet}(P)$.

- ★ $s_1 = \{A : \{A, B\}\}$

- ★ $s_2 = \{B : \{A\}\}$

- ★ $s_3 = \{B : \{A\}, A : \{\}\}$

- For a label s we define $\text{Owners}(s) = \text{Domain}(s)$

- For a security label s and principal p define

$$\text{Readers}(s, p) = \begin{cases} s(p) & \text{if } p \in \text{Owners}(s) \\ P & \text{if } p \notin \text{Owners}(s) \end{cases}$$

- Alternative notation (bit easier to read):

$$\{(A : A, C), (B : B, C)\} \text{ for } \{A : \{A, C\}, B : \{B, C\}\}$$

2. Ordering

- for two security labels s_1, s_2 we have
 - ★ $s_1 \sqsubseteq s_2$ iff
 - $\text{Owners}(s_1) \subseteq \text{Owners}(s_2)$ and
 - $\text{Readers}(s_1, o) \supseteq \text{Readers}(s_2, o)$ for every $o \in \text{Owners}(s_1)$
 - ★ $s_1 \sqcup s_2$ such that
 - $\text{Owners}(s_1 \sqcup s_2) = \text{Owners}(s_1) \cup \text{Owners}(s_2)$
 - $\text{Readers}(s_1 \sqcup s_2, o) = \text{Readers}(s_1, o) \cap \text{Readers}(s_2, o)$
for every $o \in \text{Owners}(s_1 \sqcup s_2)$
 - ★ $s_1 \sqcap s_2$ such that
 - $\text{Owners}(s_1 \sqcap s_2) = \text{Owners}(s_1) \cap \text{Owners}(s_2)$
 - $\text{Readers}(s_1 \sqcap s_2, o) = \text{Readers}(s_1, o) \cup \text{Readers}(s_2, o)$
for every owner $o \in \text{Owners}(s_1 \sqcap s_2)$

Examples:

- $\{(A : A, B)\} \sqsubseteq \{(A : A), (B : A, B)\}$
- $\{(A : A, B), (C : A, C)\} \sqcup \{(A : A, C), (B : A, B)\}$
 $= \{(A : A), (B : A, B), (C : A, C)\}$
- Write $\{\perp\}$ for the bottom element ($\sqcap$ of all labels), which is: no owners, **everybody** can read!

Label Interpretation

Useful definition that gives an intuitive explanation to our labels:

In the case of data with a **confidentiality** label s

$$\text{EffectiveReaders}(s) = \bigcap_{o \in \text{Owners}(s)} \text{Readers}(s, o)$$

Only principals in the effective readers set can read the data.

Example $\text{EffectiveReaders}(\{(B : A, B), (A : A)\}) = \{A\}$

Special case: for the bottom element $\{\perp\}$ (no owners), everybody (P) is allowed to read:

$$\text{EffectiveReaders}(\{\perp\}) = P$$

3. Declassification rule

Rule (confidentiality) An owner o can declassify their data **only** in the following ways:

- **add readers** for owner o
- or **remove the owner** o .

$L_1 = \{(A : A, B), (B : B, C, D), (C : A, B, C)\}$

Effective readers: $\{B\}$

Owner A can

- Add readers, e.g.,
 $L_2 = \{(A : A, B, C, D), (B : B, C, D), (C : A, B, C)\}$
 - ★ this makes C an effective reader, but not D because C does not support that.
- Remove itself as owner: $L_3 = \{(B : B, C, D), (C : A, B, C)\}$, thus making C an effective reader by removing the A 's constraint that only A and B can read.
 - ★ Removing yourself is equivalent to adding everybody as a reader.

4. The act as relation

Declassification is only allowed to an entity who has the right to declassify.

- We can define for each process that it can act on behalf of principals, e.g., “process X can act on behalf of hospital and patient P ”
- Think of this as a form of delegation, e.g., a patient gives the hospital the authority to use some data for some purposes.
- By default, all processes run without any authority.
- The special construct `if_acts_for(X,Y) then Z`
 - ★ checks if the current process X is allowed to assume authority Y
 - ★ and if so, executes command Z with that authority;
- Declassification can only happen in the Z block of an `if_acts_for`

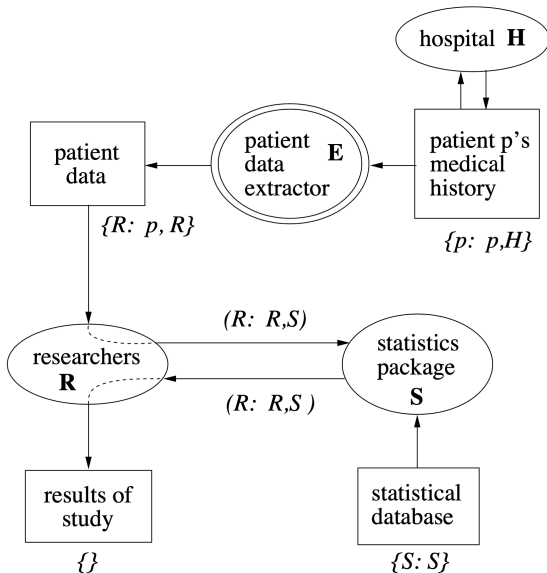
Declassification

Example: Login-Program

```
pinfo = record [name,password:string{chkr:chkr}]
check_pw(db:array[pinfo{⊥}]{⊥},
        name:string{⊥}, password:string{client:chkr})
returns ret:bool{client:chkr}
i: int{chkr:chkr} :=0;
match: bool{client:chkr,chkr:chkr} :=false;
while (i<db.length) do
  if db[i].name=name && db[i].password=password
  then match:=true
  i:=i+1
ret:=false
if_acts_for(check_pw,chkr) then
  ret:=declassify(match,{client:chkr})
```

- chkr: special authority that owns the password database.
- The check_pw tries to assume the chkr authority, and, if successful, declassifies match.

Hospital Example



Why not simply...?

Observation:

- The effective readers are the ones allowed to read by every owner.
- If $s_1 \sqsubseteq s_2$, then always $\text{EffectiveReaders}(s_1) \supseteq \text{EffectiveReaders}(s_2)$.
 - ★ so information can only legally flow from an s_1 -variable into an s_2 -variable, if s_2 is at least as restrictive on the EffectiveReaders.

Question

So why not **simplify** the labels to just the effective readers, i.e., a single set of participants who are allowed to read?

Why not simply...?

Observation:

- The effective readers are the ones allowed to read by every owner.
- If $s_1 \sqsubseteq s_2$, then always $\text{EffectiveReaders}(s_1) \supseteq \text{EffectiveReaders}(s_2)$.
 - ★ so information can only legally flow from an s_1 -variable into an s_2 -variable, if s_2 is at least as restrictive on the EffectiveReaders.

Question

So why not **simplify** the labels to just the effective readers, i.e., a single set of participants who are allowed to read?

Imagine the following scenario:

- A hospital has **some** data that can only be read by the hospital. (Not directly patient data, but **maybe** derived from it)
- Some patients have agreed that their data can be used in medical studies (after some anonymization), others have not.
- Can the data be declassified for medical studies?

Why not simply...?

Observation:

- The effective readers are the ones allowed to read by every owner.
- If $s_1 \sqsubseteq s_2$, then always $\text{EffectiveReaders}(s_1) \supseteq \text{EffectiveReaders}(s_2)$.
 - ★ so information can only legally flow from an s_1 -variable into an s_2 -variable, if s_2 is at least as restrictive on the EffectiveReaders.

Question

So why not **simplify** the labels to just the effective readers, i.e., a single set of participants who are allowed to read?

Because of declassification:

- The labels keep track of who has a say on the data.
- This avoids that we forget somebody's confidentiality rights to data when declassifying.

Integrity

Like the other information flow approaches, also Myers' can be used for integrity with the following security labels:

- Instead of Readers we have Writers
- $s_1 \sqsubseteq s_2$ iff $\text{Owners}(s_1) \supseteq \text{Owners}(s_2)$ and
 $\forall o \in \text{Owners}(s_2) : \text{Writers}(s_1, o) \subseteq \text{Writers}(s_2, o)$
- $s_1 \sqcup s_2$ s.t. $\text{Owners}(s_1 \sqcup s_2) = \text{Owners}(s_1) \cap \text{Owners}(s_2)$
 $\text{Writers}(s_1 \sqcup s_2) = \text{Writers}(s_1, o) \cup \text{Writers}(s_2, o)$
- $s_1 \sqcap s_2$ s.t. $\text{Owners}(s_1 \sqcap s_2) = \text{Owners}(s_1) \cup \text{Owners}(s_2)$
 $\text{Writers}(s_1 \sqcap s_2) = \text{Writers}(s_1, o) \cap \text{Writers}(s_2, o)$

Challenge

```
testrecord = record [ subject : cpr {shs:shs},  
                      positive: bool {shs:shs} ]  
publish_stat(db:array[testrecord:{⊥}]{⊥})  
returns infections:int{⊥}  
  
lookup_result(db:array[testrecord:{⊥}]{⊥},  
              client:cpr{⊥})  
returns result:bool{client:client}
```

shs is for sundhedsstyrelsen

Implement the functions `publish_stat` (giving total number of infections, i.e., positive testrecords) and `lookup_result` (giving the result of the specified client):

- What authority/declassifications do the functions need?
- Prove that this is indeed safe.

References I



A. C. Myers and B. Liskov.

A decentralized model for information flow control.

In M. Banâtre, H. M. Levy, and W. M. Waite, editors, *ACM Symposium on Operating System Principles*, pages 129–142. ACM, 1997.